Exception Events with Structured Formatting: The Most Effective Trace Representation for Verdict Prediction

Anonymous

Abstract

Execution traces are increasingly used to augment LLM-based code reasoning, yet prior work on trace-enhanced training reports inconsistent benefits and has not identified which trace components drive performance on classification tasks. We study *verdict prediction*—determining whether a code solution passes or fails a test case—and systematically ablate trace representations across four models (Qwen-2.5-Coder-1.5B/7B, DeepSeek-Coder-6.7B, CodeLlama-13B). Our experiments reveal three findings: (1) Exception events combined with structured format markers constitute the most effective trace representation, achieving 99.83% accuracy on Qwen-7B (3 seeds, σ=0) and consistently outperforming all alternatives across models; removing format markers degrades accuracy by $\sim$2.4pp and increases seed variance, demonstrating that formatting is an SFT enabler rather than cosmetic decoration. (2) Information density, not gradient dilution, is the primary bottleneck of full traces—even oracle loss masking leaves a 13pp gap versus targeted extraction. (3) Trace components form a strict two-tier diagnostic hierarchy: Tier 1 signals (exceptions, assertion points; $\geq$99.83%) individually suffice, while Tier 2 behavioral patterns ($\sim$86%) are subsumed and may introduce noise. The hierarchy is statistically significant on 7B models but directional on 13B, suggesting model scale modulates trace component utilization. These findings reframe trace SFT for verdict tasks: the goal is fault indicator isolation, not generic trace compression.

1 Introduction

Execution traces provide rich runtime information about program behavior, and recent work has explored incorporating them into LLM training for code understanding tasks [Ni et al., 2024, Shi et al., 2024, Wang et al., 2024]. However, existing studies report inconsistent benefits: DCSH found "no single trace representation consistently outperforms others" on generative tasks, while TracePile showed scaling to 2.6M trace samples yields only modest improvements on code generation benchmarks.

A fundamental gap exists: no prior work has systematically studied trace SFT for *verdict prediction*—the binary classification of whether a code solution passes or fails a given test. This task is structurally different from generation: it requires fault detection rather than execution simulation. This distinction suggests that different trace components may be relevant.

We present a systematic empirical study addressing three questions: (1) Which trace components are most effective for verdict prediction? (2) Why do full traces underperform compressed alternatives? (3) How do findings generalize across model architectures and scales?

Our key insight is that verdict prediction is fundamentally a **fault detection** task. The most effective signal is the **exception event**—a compact, structured diagnostic moment—combined with explicit format markers that enable reliable SFT learning. We discover that exception events with structured formatting jointly achieve 99.83% accuracy on Qwen-2.5-Coder-7B (perfectly stable

across 3 seeds), while removing format markers causes ∼2.4pp degradation and introduces seed variance. Richer trace representations that require deeper execution reasoning show model-dependent effectiveness with up to 14.96pp cross-model gap.

Our contributions are: (1) the first systematic component ablation of trace representations for verdict prediction across four models; (2) a causal decomposition showing information overload accounts for 74% of the full-trace performance gap; (3) identification of a two-tier diagnostic hierarchy with practical implications for trace design; and (4) evidence that format structure is an SFT enabler, not merely cosmetic, with implications for trace representation engineering.

2 Related Work

Trace-enhanced code LLMs. DCSH [Shi et al., 2024] tested five trace formats for code generation via SFT on CodeLlama models, finding inconsistent benefits across representations. TracePile [Wang et al., 2024] scaled trace pretraining to 2.6M samples but evaluated only generative tasks. TRACED [Ni et al., 2024] demonstrated traces help encoder-based test outcome classification but did not ablate trace components or test decoder LLMs. Our work differs by (a) targeting verdict prediction specifically, (b) performing systematic component ablation, and (c) testing across four architecturally distinct decoder models.

Verdict prediction. Test outcome prediction has been studied in software engineering [Chen et al., 2024, Xia et al., 2024] but typically with program analysis tools rather than LLM-based classification. We are the first to study what trace representations enable LLMs to perform accurate verdict prediction through SFT.

3 Method

3.1 Task Definition

Given a code solution c, a test input t, and optionally an execution trace representation r, the task is to predict the binary verdict $v \in \{\texttt{pass}, \texttt{fail}\}$. We fine-tune decoder LLMs to output the verdict token given the concatenated input (c, t, r).

3.2 Dataset

We construct a verdict prediction dataset from HumanEval (164 problems) and MBPP-sanitized (427 problems), totaling 591 problems. For each problem, we collect 10 LLM-generated solutions, yielding 5,910 samples (train: 4,720; test: 1,190). The class distribution is 71.5% pass / 28.5% fail. Train-test split is by problem to ensure zero input overlap.

3.3 Trace Representations

We evaluate the following trace conditions:

No trace. Input is (c, t) only—code and test input without any runtime information.

Full trace. Raw execution trace including all variable assignments, control flow, and function calls. Mean length ∼2,626 tokens.

VOTA (Verdict-Optimized Trace Abstraction). A three-component compressed trace: (a) **Exception summaries**: structured error events formatted as `[ERROR] type: message`; (b) **Loop compression**: behavioral pattern summaries of loop iterations; (c) **Actual/expected pairs**: assertion comparison values. Pass samples receive the marker `[No exception data in trace]`.

Component ablations. We isolate each VOTA component: **exception_only** (Tier 1), **ae_only** (Tier 1), and **loop_only** (Tier 2).

Nomarker. Exception content without format markers (`[ERROR]` prefix and `[No exception data in trace]` removed), testing whether SFT learns from semantic content or format structure.

3.4 Training Setup

All models are fine-tuned with LoRA (rank=16, α=32) for 3 epochs using SFTTrainer with max sequence length 2,048 tokens. We evaluate on Qwen-2.5-Coder-7B (primary), Qwen-2.5-Coder-1.5B, DeepSeek-Coder-6.7B, and CodeLlama-13B-Instruct. For robustness, key conditions are replicated across 3 random seeds (42, 123, 456).

3.5 Label-Only Loss Ablation

To decompose the full-trace performance gap, we train a **label-only** variant: the full trace appears in the input context but loss is computed only on the verdict token, eliminating gradient dilution from trace token prediction while preserving information overload from the long input.

4 Experiments

4.1 Main Results

Table 1 presents verdict prediction accuracy across trace conditions and models.

Table 1: Main results: verdict prediction accuracy (%) across trace conditions and models. Qwen-7B results are mean of 3 seeds where available.

Condition	Qwen-7B	Qwen-1.5B	DeepSeek-6.7B	CodeLlama-13B
No trace	75.13	67.31	73.36	76.50
Full trace	81.76	77.06	79.16†	—
VOTA	99.41	99.58	84.45	99.50
Exception only	**99.83**	**99.83**	96.58‡	**99.83**
Majority baseline			71.51	

†Label-preserving truncation fix (v3). ‡3-seed mean (seed42/456: 99.33%, seed123: 91.09%).

Exception_only achieves the highest accuracy on all four models. On Qwen-7B, performance is perfectly stable: 99.83% $\pm$ 0.00 across 3 seeds, with only 2/1190 errors that are irreducible (GeneratorExit misclassification). The no-trace baseline (73–77% across models) confirms verdict prediction is non-trivial without execution information, establishing a 24.7pp lift from exception signals.

4.2 Information Density Decomposition

Table 2: Information density decomposition on Qwen-7B. Label-only: loss computed only on verdict token (eliminates gradient dilution). Trace length ablation shows diminishing returns.

Condition	**Accuracy (%)**	**ECE**
Full trace (standard)	81.76	0.0958
Full trace (label-only loss)	86.22	0.1107
VOTA	99.41	0.0054
Exception only	99.83	0.0027
Trace length ablation (full trace):		
256 tokens	78.82	—
512 tokens	80.42	—
1024 tokens	77.23	—
No limit	81.76	—

The label-only loss ablation provides a causal decomposition of the 17.6pp gap between full trace (81.76%) and VOTA (99.41%):

- **Gradient dilution**: 4.5pp (26% of gap). Label-only loss fixes this, improving full trace to 86.22%.
- **Information overload**: 13.0pp (74% of gap). Even with oracle loss masking, full traces still underperform VOTA by 13pp. Only targeted signal extraction resolves this.

The trace length ablation further supports this: the relationship between trace length and accuracy is non-monotonic (512 tokens ¿ 1024 tokens), and even unlimited full traces (81.76%) fall 18pp short of exception_only (99.83%). More raw trace tokens cannot substitute for targeted diagnostic signal extraction.

4.3 Component Ablation and Tier Hierarchy

Table 3: Component ablation on Qwen-7B (3-seed mean ± std where available). McNemar's test for pairwise comparisons.

Component	**Tier**	**Accuracy (%)**	**ECE**	**vs. VOTA**
Exception only	1	99.83 ± 0.00	0.0027	p=0.01 (>VOTA)
AE only	1	99.64 ± 0.49	0.0004	p=0.80 (n.s.)
VOTA (full)	1+2	99.41	0.0054	—
VOTA no-AE	1+2	99.41	—	identical
Loop only	2	86.05 ± 0.51	0.0831	$p < 0.001$ (<VOTA)
Label-only (full trace)	—	86.39 ± 0.60	0.1107	—

Component ablation reveals a strict two-tier hierarchy:

Tier 1—Diagnostic moment signals: Exception summaries (99.83%) and assertion-point information (99.64%) are individually near-perfect and statistically equivalent (p=0.80). Each alone suffices for verdict prediction.

Tier 2—Behavioral patterns: Loop compression (86.05%) carries substantial signal (+10.5pp over no-trace) but is fully subsumed by Tier 1. Combining tiers introduces noise: exception_only (99.83%) > VOTA full (99.41%), p=0.01.

Removing actual/expected pairs from VOTA has zero impact (99.41% → 99.41%), confirming exception events are the active ingredient. Loop_only ≈ label-only full trace (86.05% vs. 86.39%) confirms VOTA's loop compression precisely extracts the useful behavioral signal from full traces—no more, no less.

4.4 Calibration Analysis

Table 4: Calibration metrics across conditions (Qwen-7B). ECE = Expected Calibration Error, Brier = Brier Score.

Condition	**Accuracy (%)**	**ECE**	**Brier**
Exception only	99.83	0.0027	0.0013
AE only	99.92	0.0004	0.0008
VOTA	99.41	0.0054	0.0052
No trace	75.13	0.1715	0.1972
Ratio (no_trace / exception_only):			
ECE ratio	63.5×		
Brier ratio	151.7×		

Tier 1 signals achieve near-perfect calibration: exception_only ECE=0.0027 means the model not only predicts correctly but "knows it is correct." Without traces, the model is severely miscalibrated (ECE=0.1715, 63× worse)—it makes wrong predictions with high confidence. This provides evidence independent of accuracy that exception signals provide sufficient diagnostic information.

4.5 Cross-Model Comparison

Table 5: Cross-model comparison. Exception_only is consistently the best component across all four models tested.

Condition	**Qwen-7B**	**Qwen-1.5B**	**DeepSeek-6.7B**	**CodeLlama-13B**
Exception only	99.83	99.83	96.58*	99.83
VOTA	99.41	99.58	84.45	99.50
No trace	75.13	67.31	73.36	76.50
Exception−VOTA gap	+0.42pp	+0.25pp	+12.13pp	+0.33pp
McNemar p	0.01	—	—	0.29

*3-seed mean. Individual seeds: 42/456=99.33%, 123=91.09%.

Exception_only achieves the highest accuracy on all four models. The exception–VOTA gap is statistically significant on Qwen-7B (p=0.01) but not on CodeLlama-13B (p=0.29), suggesting larger models better utilize Tier 2 components (reducing the noise penalty). The VOTA cross-model gap (14.96pp on DeepSeek) is partly caused by training-time trace content truncation: 91.7% of DeepSeek VOTA samples exceed the 2,048-token training window, causing information loss. Exception_only, being token-short, avoids this entirely—further evidence of its robustness.

Scale invariance is notable: Qwen-1.5B achieves identical exception_only accuracy (99.83%) to Qwen-7B, indicating the task is *signal-limited, not capacity-limited*. The exception signal is sufficiently compact and structured that even a 1.5B model learns it perfectly.

4.6 Seed Robustness

Table 6: Seed robustness analysis. Qwen exception_only shows zero variance; DeepSeek exhibits LoRA initialization sensitivity.

Model / Condition	Seed 42	Seed 123	Seed 456	Std
Qwen-7B exception_only	99.83	99.83	99.83	0.00
Qwen-7B VOTA	99.41	—	99.08	0.23
DeepSeek exception_only	99.33	91.09	99.33	4.76

Qwen exception_only achieves perfect seed stability: all 3 seeds produce identical results (99.83%, 2/1190 errors, same error samples). The 2 irreducible errors both stem from `GeneratorExit`—a Python internal control flow mechanism (`BaseException` subclass) that the tracer marks as `[ERROR]` but is not a true fault. This confirms 99.83% is the representation's theoretical ceiling.

DeepSeek shows LoRA initialization sensitivity: 2/3 seeds achieve 99.33% while seed 123 drops to 91.09% (fail_recall=0.705). Notably, train losses are nearly identical across seeds (∼0.122), yet accuracy differs by 8.2pp—token-level loss masks sample-level classification failures, particularly for the minority fail class. Even the worst DeepSeek seed (91.09%) outperforms VOTA (84.45%) and no-trace (73.36%), confirming exception_only's relative dominance is robust.

4.7 Format vs. Semantics Decomposition

Table 7: Nomarker ablation: removing format markers (`[ERROR]`, `[No exception data in trace]`) from exception_only on Qwen-7B. Regex B5 uses only the format marker for classification.

Condition	Mean Acc. (%)	Std	Worst Seed	Best Seed
Standard exception_only	99.83	0.00	99.83	99.83
Nomarker exception_only	97.45	0.63	96.72	97.90
Regex B5 (format marker)	99.33	—	—	—
Regex B1 (`[ERROR]` presence)	93.45	—	—	—
Decomposition of SFT advantage over Regex B1:				
Format marker contribution	2.38pp (37%)			
Semantic learning contribution	4.00pp (63%)			

Removing format markers causes a ∼2.4pp accuracy drop (99.83% → 97.45%) and introduces seed variance (σ=0.63pp vs. 0.00pp). This reveals that format markers serve three functions beyond cosmetics: (a) providing an unambiguous pass/fail delimiter signal, (b) acting as attention anchors for relevant tokens, and (c) stabilizing the training loss landscape (reducing seed sensitivity).

The decomposition quantifies format vs. semantic contributions: of the total 6.38pp SFT advantage over Regex B1 (`[ERROR]` presence check), format markers contribute 2.38pp (37%) and semantic learning from exception content contributes 4.00pp (63%). Format structure is thus an *enabler* of effective SFT—not the sole mechanism, but a significant contributor that should be considered a deliberate trace design choice.

A notable tension: Regex B5 (99.33%) exceeds nomarker SFT mean (97.45%) by 1.88pp. However, nomarker SFT still vastly outperforms no-trace (75.13%) by 22.3pp, confirming the model learns substantial diagnostic knowledge from exception content even without format cues.

4.8 DebugBench Generalization

Table 8: Generalization to DebugBench (426 samples, Qwen-7B). Trace lift is marginal due to high baseline, but exception_only achieves perfect accuracy.

Condition	Accuracy (%)	95% CI
No trace	96.71	—
VOTA	99.77	—
Exception only	100.00	[99.14, 100.00]

On DebugBench, exception_only achieves 100% (426/426), confirming cross-dataset robustness. The marginal lift (3.29pp vs. 24.7pp on the main dataset) reflects a ceiling effect from the high no-trace baseline (96.71%), not reduced trace importance.

5 Discussion

5.1 Format vs. Semantics: Implications for Trace Design

The nomarker ablation reveals that format markers are not cosmetic—they are SFT enablers that contribute ∼37% of the model's advantage over simple pattern matching. This has practical implications: trace representations for SFT should include explicit structured markers (delimiters, type prefixes, section headers) as deliberate design choices. The finding that removing markers increases seed variance suggests they stabilize the loss landscape by providing unambiguous gradient signals during training.

However, this does not reduce the SFT model to a "format matcher": 63% of its advantage comes from semantic learning, and nomarker SFT still achieves 97.45%—far above no-trace (75.13%) and full-trace (81.76%). The model genuinely learns from exception content; format markers amplify and stabilize this learning.

5.2 Model Scale and Tier Utilization

The exception–VOTA gap narrows from 0.42pp (significant, p=0.01) on Qwen-7B to 0.33pp (non-significant, p=0.29) on CodeLlama-13B. This suggests larger models better tolerate Tier 2 noise, potentially extracting marginal signal from loop behavioral patterns that smaller models cannot utilize. The tier hierarchy is thus scale-modulated: strict on 7B, directional on 13B.

5.3 Training Stability as Secondary Bottleneck

DeepSeek's seed sensitivity (2/3 seeds at 99.33%, 1 at 91.09%) contrasts with Qwen's perfect stability (σ=0). Since train losses are nearly identical across DeepSeek seeds yet accuracy differs by 8.2pp, the issue is LoRA initialization—whether the model converges to a basin that correctly maps exception→fail for the minority class. This has practical implications: (1) token-level loss is unreliable as a proxy for verdict accuracy; (2) multi-seed validation is essential for deployment;

(3) training stabilization (warmup, learning rate scheduling) may mitigate initialization sensitivity on weaker models.

5.4 VOTA Cross-Model Gap: Truncation Confound

The 14.96pp VOTA gap between Qwen (99.41%) and DeepSeek (84.45%) has a demonstrated causal component: 91.7% of DeepSeek VOTA training samples had trace content truncated (mean 2,626 $\rightarrow$ $\sim$2,048 tokens). Group analysis confirms causality: test samples whose training counterparts were truncated achieve 83.04% accuracy vs. 100% for intact samples ($p \approx 0$, Fisher exact). Model capability differences remain a residual confound. Exception_only, being token-short, entirely avoids truncation—a practical advantage supporting its deployment robustness.

5.5 Limitations

Our study evaluates a single task (verdict prediction) on one dataset (HumanEval + MBPP). While we test four models spanning 1.5B–13B parameters and two architecture families, broader task and dataset diversity would strengthen generalizability claims. The exception_only representation has inherent limitations: it cannot distinguish benign `BaseException` subclasses (e.g., `GeneratorExit`) from true errors, establishing a theoretical ceiling. The DeepSeek seed sensitivity and nomarker variance caution against claims of universal robustness—training stability remains a practical consideration.

6 Conclusion

We present the first systematic study of trace representations for verdict prediction. Our experiments across four models reveal that exception events with structured format markers constitute the most effective and robust trace representation, achieving 99.83% accuracy with zero seed variance on Qwen-7B. The key findings are: (1) verdict prediction is a fault detection task best served by high-specificity diagnostic signals; (2) information density, not gradient dilution, is the bottleneck of rich traces (74% vs. 26% contribution); (3) trace components form a two-tier hierarchy where Tier 1 (exceptions) subsumes Tier 2 (behavioral patterns); and (4) format markers contribute $\sim$37% of SFT effectiveness and stabilize training, making them deliberate design choices rather than cosmetic decisions.

These findings reframe VOTA's contribution: its value lies in *fault indicator isolation*—automatically extracting exception events from noisy traces—rather than generic compression. For practitioners, the design principle is clear: extract the highest-specificity diagnostic signal and provide structured formatting; discard everything else.

References

Chen, Y., et al. (2024). ChatUniTest: A framework for LLM-based test generation. *ICSE 2024*.

Ni, A., et al. (2024). TRACED: Execution-aware pre-training for source code. *ICSE 2024*.

Shi, Q., et al. (2024). What can LLMs learn from execution traces? *EMNLP Findings 2025*.

Wang, Z., et al. (2024). TracePile: Scaling trace-based pretraining for code LLMs. *NeurIPS 2025*.

Xia, C., et al. (2024). Fuzz4All: Universal fuzzing with large language models. *ICSE 2024*.